TABLA RESUMEN INSTRUCCIONES BÁSICAS ARM7DMI-S

PROCESAMIENTO DE DATOS

Aritméticas

Nombre		Descripción	Sintaxis
ADD	Add	Rd ←Rn+<Oprnd2>	ADD{cond}{S} Rd, Rn, <Oprnd2>
ADC	Add with Carry	Rd ←Rn+<Oprnd2>+C	ADC{cond}{S} Rd, Rn, <Oprnd2>
SUB	Substract	Rd ←Rn+\<Oprnd2>+1	SUB{cond}{S} Rd, Rn, <Oprnd2>
SBC	Subtract with Carry	Rd ←Rn+\<Oprnd2>+C	SBC{cond}{S} Rd, Rn, <Oprnd2>
RSB	Reverse Subtract	Rd ←<Oprnd2>+\Rn+1	SBC{cond}{S} Rd, Rn, <Oprnd2>
RSC	Reverse Subtract with Carry	Rd ←<Oprnd2>+\Rn+C	SBC{cond}{S} Rd, Rn, <Oprnd2>
MUL	Multiply	Rd ←Rm·Rs	MUL{cond}{S} Rd, Rm, Rs
MLA	Multiply Accumulate	Rd ←Rm·Rs+Rn	MLA{cond}{S} Rd, Rm, Rs, Rn
UMULL	Multiply Unsigned Long	RdHi|RdLo ←Rm·Rs	MLA{cond}{S} RdLo, RdHi, Rm, Rs
UMLAL	Multiply Unsigned Accumulate Long	RdHi|RdLo ←Rm·Rs+ RdHi|RdLo	MLA{cond}{S} RdLo, RdHi, Rm, Rs
SMULL	Multiply Signed Long	RdHi|RdLo ←Rm·Rs	MLA{cond}{S} RdLo, RdHi, Rm, Rs
SMLAL	Multiply Signed Accumulate Long	RdHi|RdLo ←Rm·Rs+ RdHi|RdLo	MLA{cond}{S} RdLo, RdHi, Rm, Rs

Lógicas

Nombre		Descripción	Sintaxis
AND	AND	Rd ←Rn AND <Oprnd2>	AND{cond}{S} Rd, Rn, <Oprnd2>
EOR	XOR	Rd ←Rn XOR <Oprnd2>	EOR{cond}{S} Rd, Rn, <Oprnd2>
ORR	OR	Rd ←Rn OR <Oprnd2>	ORR{cond}{S} Rd, Rn, <Oprnd2>
BIC	AND \	Rd ←Rn AND \<Oprnd2>	BIC{cond}{S} Rd, Rn, <Oprnd2>

Comparación

Nombre		Descripción	Sintaxis
CMP	Compare	CPSR_f (Rn+\<Oprnd2>+1)	CMP{cond} Rn, <Oprnd2>
CMN	Compare Negative	CPSR_f (Rn+<Oprnd2>)	CMN{cond} Rn, <Oprnd2>
TST	Test	CPSR_f (Rn AND <Oprnd2>)	TST{cond} Rn, <Oprnd2>
TEQ	Test Equivalence	CPSR_f (Rn XOR <Oprnd2>)	TEQ{cond} Rn, <Oprnd2>

DE TRANSFERENCIA

Entre Registros

Nombre		Descripción	Sintaxis
MOV	Move	Rd ← <Oprnd2>	MOV{cond}{S} Rd, <Oprnd2>
MVN	Move NOT	Rd ← \<Oprnd2>	MVN{cond}{S} Rd, <Oprnd2>
MRS	Move PSR to Reg	Rd ← PSR	MRS{cond} Rd, PSR
MSR	Move Reg to PSR	PSR ← Rm PSR ← #32bit_Inm	MSR{cond} PSR{field}, Rm MSR{cond} PSR{field}, #32bit_Inm

Entre registros ↔ Memoria (Simples)

Nombre		Descripción	Sintaxis
LDR	Load Word	Rd ← M[<a_mode2(Rn)>]	LDR{cond} Rd, <a_mode2(Rn)>
LDRB	Load Byte	Rd ← M[<a_mode2(Rn)>]	LDR{cond}B Rd, <a_mode2(Rn)>
LDRSB	Load Byte Signed	Rd ← M[<a_mode2(Rn)>]	LDR{cond}SB Rd, <a_mode2(Rn)>
LDRH	Load Half-Word	Rd ← M[<a_mode2(Rn)>]	LDR{cond}H Rd, <a_mode2(Rn)>*
LDRSH	Load Half-Word Signed	Rd ← M[<a_mode2(Rn)>]	LDR{cond}SH Rd, <a_mode2(Rn)>*
STR	Store Word	M[<a_mode2(Rn)>] ← Rd	STR{cond} Rd, <a_mode2(Rn)>
STRB	Store Byte	M[<a_mode2(Rn)>] ← Rd	STR{cond}B Rd, <a_mode2(Rn)>
STRH	Store Half-Word	M[<a_mode2(Rn)>] ← Rd	STR{cond}H Rd, <a_mode2(Rn)>*
SWP	Swap register with memory	Rd ← M[Rn], M[Rn] ← Rm	SWP{cond} Rd, Rm, [Rn]
SWPB	Swap Byte	Rd ← M[Rn], M[Rn] ← Rm	SWP{cond}B Rd, Rm, [Rn]

Entre registros ↔ Memoria (Múltiples)

Nombre		Descripción	Sintaxis
LDMIB LDMED	Load multiple Increment before	!: Rd ← Rd+… <reglist>←M[Rd]	LDM{cond}IB Rd{!},<reglist>{^}
LDMIA LDMFD	Load multiple increment after	<reglist>←M[Rd] !: Rd ← Rd+…	LDM{cond}IA Rd{!},<reglist>{^}
LDMDB LDMEA	Load multiple decrement before	!: Rd ← Rd-… <reglisti>←M[Rd]	LDM{cond}DB Rd{!},<reglist>{^}
LDMDA LDMFA	Load multiple decrement after	<reglist>←M[Rd] !: Rd ← Rd-….	LDM{cond}DA Rd{!},<reglist>{^}
STMIB STMFA	Store multiple increment before	!: Rd ← Rd+… M[Rd] ← <regist>	STM{cond}IB Rd{!},<reglist>{^}
STMIA STMEA	Store multiple increment after	M[Rd] ← <reglist> !: Rd ← Rd+…	STM{cond}IA Rd{!},<reglist>{^}
STMDB STMFD	Store multiple decrement before	!: Rd ← Rd-… M[Rd] ← <reglist>	STM{cond}DB Rd{!},<reglist>{^}
STMDA STMED	Store multiple decrement after	M[Rd] ← <reglist> !: Rd ← Rd-…	STM{cond}DA Rd{!},<reglist>{^}

DE CONTROL DE FLUJO DE PROGRAMA (SALTOS)

Nombre		Descripción	Sintaxis
B	Branch	PC← label (±32Mb)	B{cond} label
BL	Branch with link	LR←PC, PC← label(±32Mb)	BL{cond} label
BX	Branch and Exchange Inst. Set	PC←Rn, T_bit (CPRS)←Rn[0]	BX{cond} Rn
SWI	Software Interrumpt	SWI{cond} 24bit_Inm	OS call

TABLA RESUMEN INSTRUCCIONES BÁSICAS ARM7DMI-S

{}	Optional element
<>	compulsory operand
[]	Memory Address
#	Immediate Operand
\	NOT (logic)
S	Sets condition codes
C	carry bit of CPRS
PSR	either CPSR or SPSR
#32bit_Inm	a 32-bit constant, formed by right-rotating an 8-bit value by an even number of bits (presentation 26,27)
field	See Table 1
cond	See table 2
<Oprnd2>	See table 3
<a_mode2(Rn)>	See Table 4
^	if PC is in the <reglist> CPRS is restored with SPRS
!	Base register is updated

Pseudoinstructions

Syntax	Function
LDR Rd, =Inm	Load Inm in Rd Rd← Inm (32bit_Inm)
ADR Rd, label	Load Rd with the address of label Rd← label

Table 1 – Fields {field}

Suffix	Sets
_c	Control field mask bit
_f	Flags field mask bit
_s	Status field mask bit
_x	Extension field mask bit

Table 2 – Condition Field {cond}

Suffix	Description	Flags tested
EQ	Equal	Z=1
NE	Not Equal	Z=0
CS/HS	Unsigned higher, or same	C=1
CC/LO	Unsigned lower	C=0
MI	Negative	N=1
PL	Positive, or zero	N=0
VS	Overflow	V=1
VC	No overflow	V=0
HI	Unsigned higher	C=1 & Z=0
LS	Unsigned lower, or same	C=0 or Z=1
GE	Greater, or equal	N=V
LT	Less than	N!=V
GT	Greater than	Z=0 & N=V
LE	Less than, or equal	Z=1 or N=!V
AL	Always	

Table 3 – Operand 2 <Oprnd2>

Operation	Assembler
Immediate value	#32bit_Inm
Logical shift left	Rm, LSL #5bit_Inm
Logical shift right	Rm, LSR #5bit_Inm
Arithmetic shift right	Rm, ASR #5bit_Inm
Rotate right	Rm, ROR #5bit_Inm
Register	Rm
Logical shift left	Rm, LSL Rs
Logical shift right	Rm, LSR Rs
Arithmetic shift right	Rm, ASR Rs
Rotate right	Rm, ROR Rs
Rotate right extended	Rm, RRX

Table 4 - <a_mode2(Rn)>

Operation	Assembler
Immediate Offset	[Rn, #+/-12bit_Offset]
Register Offset	[Rn, +/- Rm]
Scaled Register Offset	[Rn, +/-Rm, LSL #5bit_shift_imm] [Rn, +/-Rm, LSR #5bit_shift_imm] [Rn, +/-Rm, ASR #5bit_shift_imm] [Rn, +/-Rm, ROR #5bit_shift_imm] [Rn, +/-Rm, RRX]
Pre-indexed immediate offset	[Rn, #+/-12bit_Offset]!
Pre-indexed register offset	[Rn, +/-Rm]!
Pre-indexed scaled register offset	[Rn, +/-Rm, LSL #5bit_shift_imm]! [Rn, +/-Rm, LSR #5bit_shift_imm]! [Rn, +/-Rm, ASR #5bit_shift_imm]! [Rn, +/-Rm, ROR #5bit_shift_imm]! [Rn, +/-Rm, RRX]!
Post-indexed immediate offset	[Rn], #+/-12bit_Offset
Post-indexed register offset	[Rn], +/-Rm
Post-indexed scaled register offset	[Rn], +/-Rm, LSL #5bit_shift_imm [Rn], +/-Rm, LSR #5bit_shift_imm [Rn], +/-Rm, ASR #5bit_shift_imm [Rn], +/-Rm, ROR #5bit_shift_imm [Rn], +/-Rm, RRX

*with signed byte and halfword data transfers only immediate (unsigned 8-bit constant) or register offset (without shift) are possible (with non- or pre- or post- indexed options)

[Rn{]}, #+/-8bit_Offset{]}{!}

[Rn{]}, +/-Rm{]}{!}